

Cryptography for Non-Math People – Michael Waterman

 michaelwaterman.nl/2026/01/15/cryptography-for-non-math-people

Michael Waterman

January 15, 2026



Over the years I've written quite a bit about cryptography. [PKI](#), certificates, trust chains, identity, and even a deep dive into [Diffie-Hellman key exchange](#). All fairly technical topics, and topics I genuinely enjoy writing about. Yet there was always something missing.

I never really wrote about the *basics*, not because they aren't important, but because they are often explained in a way that immediately scares people off, including myself. As soon as cryptography shows up, so do formulas, symbols, and the implicit message that you need to be "good at math" to understand any of it. I've heard this countless times from students and professionals alike:

| *"I understand how to use cryptography, but I don't really understand how it works."*

And often followed by:

| *"I was never good at math."*

Here's the thing, that sentence "*I'm not good at math*" is usually irrelevant. Cryptography *uses* mathematics, but understanding cryptography is not about solving equations. It's about understanding systems, assumptions, and one-way relationships. The math exists to guarantee certain properties, not to be memorized or reproduced by hand.

Once I stopped treating cryptography as "math I don't fully remember" and started treating it as "a system with very specific rules", things became much clearer, even concepts I

thought I already understood. This article is the result of that shift in perspective, hope you enjoy the read.

For this blog post I would like to do a shout out to some wonderful people that have offered their help during my research, writing and validation.

Thanks, [Jake Hildreth](#), [Max Eggens](#) and [Julie Rubens](#), *couldn't have done it without you!*

What cryptography tries to solve

Before talking about keys, algorithms, or key sizes, it helps to zoom out. Cryptography is not about hiding information for the sake of secrecy. It exists to solve a few very concrete problems that show up everywhere in modern (and ancient) IT. At its core, cryptography tries to provide answers to four questions, which happens to be the core of security:

Confidentiality

Who is allowed to read this information? If I send data over the internet, how do I make sure that only the intended recipient can understand it, even if others can see the traffic?

Integrity

Has this information been modified? How do I know that what I received is exactly what was sent, without subtle changes along the way?

Non-repudiation

Can someone deny they did this? How do we make sure a sender can't later claim "I never sent that," or a signer can't say "that's not my approval," once the action has real consequences?

Authenticity

Who is this really coming from? How can I be confident that I'm talking to the real server, user, or service, and not an impostor?

Most systems we trust daily rely on these guarantees:

- HTTPS connections
- Software updates
- Authentication systems
- Encrypted messaging

Cryptography doesn't prevent attackers from seeing traffic or trying to interfere. It makes sure that even if they do, the results are useless or immediately detectable. This framing is

important, because it explains why cryptography is designed the way it is, and why certain trade-offs exist. Easy, right, let's talk crypto next.

Symmetric cryptography

Almost all modern cryptographic systems use two different types of cryptography, each with its own role. Let's start with, Symmetric cryptography. Symmetric cryptography is the simplest model to understand.

- There's one shared secret key
- The same key is used to encrypt and decrypt

If you and I both know the same secret, we can communicate securely. This type of cryptography is:

- Very fast
- Very efficient
- Ideal for encrypting large amounts of data

The problem isn't how well it works, the problem is key distribution. How do you safely share that secret key in the first place, especially over an untrusted network? Let's take a look at a concrete example. Imagine you and I agree on a shared secret. Not a password in the login sense, but an actual secret rule that only the two of us know.

For example:

"Every letter in the message is shifted three positions forward in the alphabet."

So:

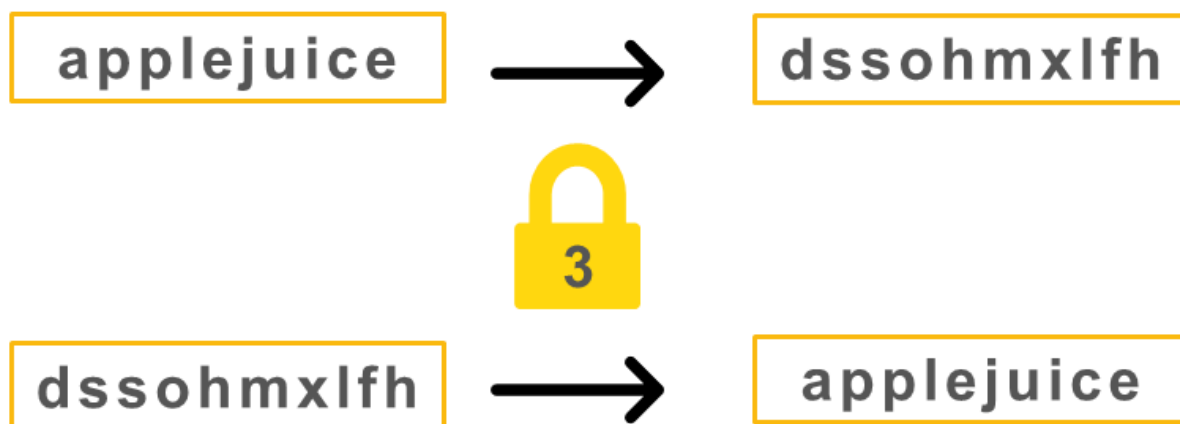
- A becomes D
- B becomes E
- C becomes F

If I send you the word: "*Hello*", I apply our shared rule and send, "*Khoor*", Anyone can see the message. But without knowing the rule, it's meaningless. Because you know the same secret rule, you simply reverse it and recover the original message.

Did you know that this form of symmetric encryption is very old? Old as in *more than 2,000 years old*. This simple substitution of letters is commonly known as the Caesar cipher, named after Julius Caesar, who reportedly used it during his military campaigns in Gaul. Messages had to be delivered by runners, for obvious reasons, digital communication hadn't quite been invented yet.

If one of those runners was captured by the enemy, the message itself would still be readable, but meaningless without knowing the substitution rule. As long as both sender and receiver knew the same rule, the message could be encrypted and decrypted with ease. This is symmetric encryption in its purest form, the same secret is used on both sides. Any Caesar wasn't even the first, there are many documented examples of using symmetric encryption in the ancient past.

Caesar Cipher - Substitute



The idea is ancient, but the principle hasn't changed. Modern symmetric algorithms like AES (Used in BitLocker for example) work on exactly the same assumption, the algorithm can be public, but the shared secret must remain secret.

This is symmetric cryptography in its purest form:

- One shared secret
- The same secret to encrypt and decrypt

A simple intuition for modern symmetric encryption

Thus far, we've used a letter substitution to explain symmetric encryption. While modern algorithms don't work on letters anymore, the underlying idea is remarkably similar, data is mixed with a secret in a way that can be reversed, as long as you know that same secret. A very simple way to illustrate this idea is with an operation called XOR. Don't worry about bits or binary math here. The only thing that matters is its key property: applying XOR twice with the same secret brings you back to where you started.

XOR in, XOR out, just like in the Karate Kid..... sigh, I'm getting old. In other words, XOR doesn't just encrypt, it also decrypts.

A tiny XOR example

Imagine a message represented as a number:

Message = 6
Secret = 3

To encrypt the message, we combine it with the secret:

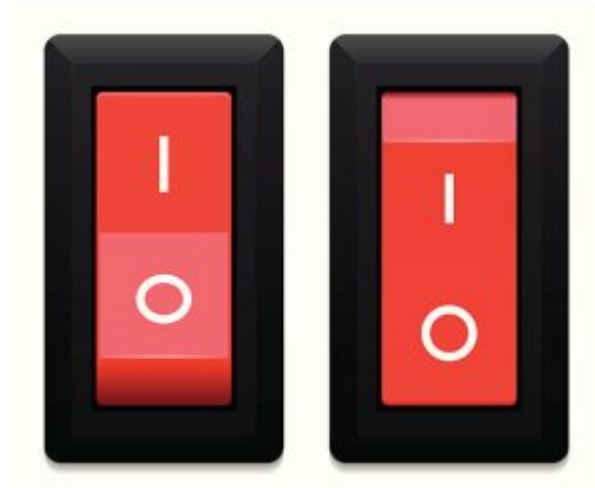
$6 \text{ XOR } 3 = 5$

The result looks different from the original message, it's scrambled. Now comes the important part. To decrypt the message, we don't undo anything. We simply apply the same operation again, using the same secret:

$5 \text{ XOR } 3 = 6$

And we're back where we started. The original message is recovered, not because we reversed the operation, but because XOR is its own inverse. The same secret both hides and reveals the data.

A helpful way to think about XOR is a light switch. Flip the switch once, and the state changes. Flip the same switch again, and you're back where you started. XOR works in exactly the same way. Applying XOR with a secret once scrambles the data. Applying XOR again with that same secret restores the original data. The operation undoes itself.



Switch in the on and off position

Modern symmetric algorithms like AES are far more complex than XOR, but this fundamental idea remains the same. A shared secret is used to transform data in a reversible way. As long as both sides know that secret, encryption and decryption are straightforward. And this is exactly why symmetric encryption works so well, and why securely sharing that secret becomes the real challenge.

From XOR to real-world symmetric encryption

The XOR example shows the core idea behind symmetric encryption: data is combined with a secret in a reversible way. In practice, modern encryption algorithms build on this idea in two slightly different ways.

Stream ciphers: encrypting bit by bit

Stream ciphers work very much like the XOR example you just saw. They generate a stream of pseudo-random bits based on a secret key, and combine that stream with the data, typically using XOR. You can think of this as encrypting the data bit by bit, as it flows past. As long as both sides generate the same key stream, applying the same operation again recovers the original data. Algorithms like ChaCha20 work in this way. They are fast, efficient, and especially well-suited for environments where data arrives as a continuous stream, such as network traffic.

Block ciphers: encrypting in chunks

Block ciphers, like we use in disk encryption take a different approach. Instead of processing data bit by bit, they work on fixed-size blocks of data, for example, 128 bits at a time.

AES is the most well-known example. For people working with Microsoft Windows, BitLocker is a good example of where this is used. Internally, it still relies on reversible transformations and mixing operations, but it applies them to entire blocks rather than individual bits. Additional constructions, called modes of operation, define how those blocks are chained together to securely encrypt larger amounts of data. The result is a system that is extremely robust, widely analyzed, and suitable for encrypting large volumes of data, such as disks or files.

A real-world problem: symmetric encryption and memory

On paper, modern symmetric encryption is extremely strong. Algorithms like AES have been studied for decades and are not considered broken in any meaningful way. And yet, real-world systems that rely on symmetric encryption can still be compromised, not by breaking the math, but by attacking how the keys are handled. A good example of this is full-disk encryption. Technologies like BitLocker use symmetric encryption to protect the contents of a disk. The data on disk is encrypted with a secret key, and as long as that key remains secret, the data is unreadable. But there is an unavoidable reality, at some point, the system must be able to use that key. And to use it, the key must exist in memory.

| *“This is where the weakness appears.”*

Over the years, researchers have demonstrated that if an attacker gains sufficient access to a running or suspended system, it may be possible to extract encryption keys directly from memory. Once that key is recovered, the encryption itself offers no further protection, the disk can simply be decrypted as intended. Nothing was cracked, nothing was brute-forced. The system behaved exactly as designed. This highlights an important lesson about symmetric encryption, the algorithm can be perfectly secure, while the system around it is not. Symmetric encryption assumes that the secret key can be kept secret at all times. In practice, that assumption is much harder to uphold, especially on general-purpose systems where memory, firmware, and hardware all become part of the attack surface.

| *“In other words, the weakest link is rarely the cipher. It’s where the key lives.”*

The same difference

Despite their differences, substitution ciphers, stream ciphers and block ciphers share the same fundamental principle, a shared secret is used to transform data in a way that only someone with that same secret can undo. The mechanics differ, but the trust model remains identical. And once again, we arrive at the same question:

How do both sides get that shared secret in the first place? Which brings us right back to asymmetric cryptography.

From symmetric to asymmetric cryptography

At this point, symmetric cryptography should feel fairly intuitive.

It works well, it’s fast, and conceptually it’s simple:

- Both parties share the same secret
- The algorithm can be public
- The secret key is the only thing that must remain hidden

So why do we need anything else? The answer lies entirely in one uncomfortable question:

| *How do two parties safely agree on a shared secret if they’ve never communicated securely before?*

This is known as the key distribution problem, and it’s the reason asymmetric cryptography exists at all.

The key distribution problem

Imagine I want to send you an encrypted message using symmetric encryption. Before I can do that, we need to agree on a secret key. But we are faced with the chicken and the

egg problem, what comes first? Are these valid ways of sending the secret:

- Email is observable
- Networks are shared
- Pigeons can be captured in flight.
- Attackers don't need to block traffic, only observe it

If I send you the secret key first, an attacker can simply copy it, and now they can decrypt everything we exchange afterwards. Symmetric cryptography assumes something very strong:

A secure channel already exists.

Asymmetric cryptography exists to create that secure channel *without* already having one. Cool stuff coming up!

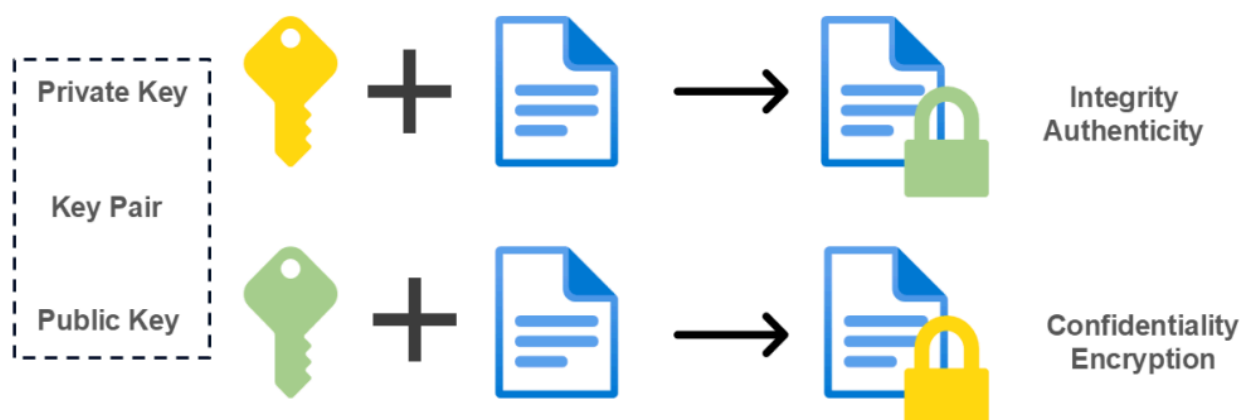
The core idea behind asymmetric cryptography

Asymmetric cryptography introduces a new concept: key pairs. Instead of one shared secret, we now have:

- A public key, safe to distribute (It's like Oprah said, "everybody gets one"!)
- A private key, must remain secret (The core essence)

These two keys are *mathematically related*, but remember this:

- Knowing the public key does not allow you to derive the private key
- What one key does, only the other key can undo, look at it as a lock and key, they belong to each other.



This enables something fundamentally new. I can publish my public key anywhere:

- On a website

- In a certificate
- In a directory
- On a talk show (*Yes, this is a joke, but you actually can do that*)

Anyone can use that public key to encrypt data *for me* (The lock). Only I, the holder of the private key, can decrypt it. This single idea solves the key distribution problem.

Something to clarify

A common misunderstanding is that a public key is just a hidden or scrambled version of the private key. It's not, let me clarify. Think of it like a lock and a key.

The public key is the lock: you can hand it out to anyone.

The private key is the key that opens it: only the owner should have it.

The lock is not a “hidden version” of the key. They are different things, designed to work together. Yet, both are forged from the same metal.

In cryptographic terms, this means:

“Two different values derived from the same origin, designed to work together in one direction only.”

Just like a lock and key forged from the same chunk of metal, they belong together, but they are not interchangeable. You can use the lock without the key, but you cannot recreate the key from the lock.

This one-way relationship is what makes public key cryptography secure.

Building intuition before the math

Before diving into any calculations, it's important to understand *what* (not how) we're trying to achieve.

We want a system where:

1. Two keys are generated together, the same source concept
2. One key can be made public
3. The other key must remain secret
4. Operations performed with one key can only be reversed with the other
5. Reversing the relationship is computationally infeasible, not impossible

RSA is one of the oldest and most well-known systems that achieves this, and it does so using surprisingly simple ideas. To understand those ideas, we'll deliberately use very small numbers.

This example will be:

- Completely insecure
- Trivial to break
- Useless in practice
- But awesome to understand!

And that's exactly what makes it useful for learning.

RSA in a nutshell

RSA is an asymmetric cryptographic algorithm named after its inventors *Rivest, Shamir, and Adleman*, who published it in 1977 (*I just realized we're almost the same age!*). It was one of the first practical public-key systems and played a major role in making secure communication on the internet possible. RSA is based on the mathematical difficulty of factoring the product of two large prime numbers. For decades it was the default choice for key exchange and digital signatures. While still widely understood and used, modern systems increasingly prefer alternatives that offer better performance and smaller key sizes. Yet, it makes a great example to understand the basics of cryptography.

A tiny RSA example (for non-math people)

Let's build a miniature RSA system step by step. Not to implement it, certainly not to use it, but to understand why public and private keys work at all... and have a little fun while we're at it, I promise.

About prime numbers

But first, we need to talk briefly about prime numbers. And just to get this out of the way up front: no, we're not talking about Optimus Prime.

In mathematics, a prime number is simply a number that can only be divided by 1 and itself. That's it. No hidden meaning, no special trick. Numbers like 2, 3, 5, 7, and 11 are prime (Well, that's just Prime, I'll stop with the puns!). Numbers like 4, 6, 8, or 12 are not, they can be broken down into smaller factors. Take 4 for example. Besides 1 and itself, it can also be divided by 2, which means it does not qualify as a prime number.

A useful way to think about prime numbers is as Lego blocks. Any non-prime number can be constructed by multiplying primes together, but prime numbers themselves can't be broken down any further. They're the indivisible pieces. This property is exactly why they matter in cryptography.

And here's the magic, multiplying prime numbers together is easy. Figuring out which prime numbers were multiplied together is not. Well, obviously when the numbers are large

enough.

RSA takes advantage of that asymmetry. It doesn't rely on secrecy or clever tricks, it relies on the fact that reversing this process becomes infeasible, but not impossible, once the numbers grow large enough. So whenever you see RSA talking about "*choosing two primes*", remember. This isn't about obscure mathematics. It's about deliberately picking numbers that are easy to combine, but extremely hard to separate again.

Prime numbers are simple: if a number has more than two divisors, it's not prime.

Modulus explained

And there's this modulus thing. If you never had this at school, A helpful way to think about the modulus operation is to imagine a clock. A clock doesn't count endlessly upward. It has a fixed number of positions. Once you reach the end, you simply start again from the beginning. If it's 10 o'clock and you move forward 5 hours, you don't end up at 15, you end up at 3. You've gone around the clock once, and what matters is what's left after that full round. That's exactly how modulus works.

When we say "mod n ", we're saying, we only care about what remains after making as many full rounds of size n as possible. You can go around the circle once, twice, or a hundred times, only the remainder matters in the end.

"Modulus isn't about how far you go, it's about what remains."

Overview of the players

Before we continue with a Prime, here's an overview of all the symbols we will be using and what it means. Use it as a reference for the example I will be using.

Symbol	Name	Meaning
p	Prime number	One of the two secret prime numbers used to build the RSA system
q	Prime number	The second secret prime number
n	Modulus	The product of p and q ($n = p \times q$), defines the size of the key and the numeric space RSA operates in
$\phi(n)$	Euler's totient	A value derived from p and q that defines the mathematical "working space" for the key pair
e	Public exponent	Part of the public key, used for encryption or signature verification
d	Private exponent	Part of the private key, mathematically linked to e and used for decryption or signing
m	Message	The original plaintext message (represented as a number)
c	Ciphertext	The encrypted message after applying RSA

Step 1 – Choose two random prime numbers

We start by choosing two prime numbers:

$p = 3$
 $q = 11$

In real systems, these numbers are hundreds or thousands of bits long. Here, they're small so we can follow what's happening.

Step 2 – Compute n (this is the "magic number")

Next, we multiply them:

$n = p \times q$
 $3 \times 11 = 33$

When we talk about RSA key length, we're not talking about how many numbers exist, or how precise a key is. We're talking about the size of the number n , the modulus created by multiplying the two prime numbers p and q .

In practice, an RSA key size (for example 2048 bits) means that n fits into a numerical space of roughly 2^{2048} possible values. The two primes that form n are therefore each

roughly half that size, not exactly, but close enough to define the same space. This is why key length is best understood as a *range or playground*, not as an exact value you ever hit or choose explicitly.

Security doesn't come from landing on "the right" 2048-bit number. It comes from operating inside a space so large that reversing n back into its original primes becomes computationally infeasible. The larger that space, the harder the problem. To be honest, I never knew this before I did my research.

So remember, this value, n , becomes part of both the public and private key. It defines the entire numeric "space" in which RSA operates. Everything else in the system depends on this value.

Step 3 – Compute $\phi(n)$

Now we calculate a value based on p and q :

$$\begin{aligned}\phi(n) &= (p - 1)(q - 1) \\ \phi(33) &= 2 \times 10 = 20\end{aligned}$$

ϕ , which is pronounced as Phi (No, not the cake), this value must remain secret. Why? Because knowing $\phi(n)$ makes it possible to compute the private key. Mentally store it as Phi you want to eat all by yourself, never ever give it to anyone else. At this stage, the system already has an important asymmetry:

- n can be public
- p , q , and $\phi(n)$ must remain secret

What's up with the "-1"?

The "-1" in $(p - 1)(q - 1)$ is not something RSA invented. It comes from a fundamental property of prime numbers. For any prime number p , all numbers smaller than p (from 1 up to $p - 1$) share no common factors with it. In other words, for a prime number, there are exactly $p - 1$ numbers that "fit" cleanly into the mathematical space RSA needs. The same applies independently to q .

RSA simply combines those two properties. By multiplying $(p - 1)$ and $(q - 1)$, it defines the exact space in which the public and private exponents can become mathematical inverses of each other. That's why the "-1" is essential, not as a trick, but as a consequence of using prime numbers in the first place.

| *"The "-1" doesn't come from RSA, RSA takes advantage of it"*

Step 4 – Choose a public exponent (e)

Now we choose a number that:

- is smaller than $\phi(n)$
- has no common factors with $\phi(n)$

For this example:

$$e = 3$$

This value becomes part of the *public key*. The public exponent e must live inside the mathematical space defined by $\phi(n)$, and it must be positioned in such a way that it can be perfectly “undone”. If e shares any factors with $\phi(n)$, that undo operation simply doesn’t exist. All you need to remember is that it needs to be smaller to fit and function.

Step 5 – Compute the private key (d)

In math our private key would be called an exponent, but let’s simplify it to be our private key. Actually it’s the important relationship we have in this case. We need to find a number d such that:

$$(e \times d) \bmod \phi(n) = 1$$

Trying small values:

- $3 \times 7 = 21$
- $21 \bmod 20 = 1$

So: $d = 7$

Note! *The value 1 above is not chosen randomly, it has a very specific purpose. It represents the identity, the point at which encryption and decryption cancel each other out completely. The result needs to be the same, which is why the equation resolves to one: multiplying by one leaves a value unchanged.*

Think of it like Neo in The Matrix: in a world of digits, are you a zero... or the One?

The resulting key pair

At this point we have:

- *Public key:* ($n = 33$, $e = 3$)
- *Private key:* ($n = 33$, $d = 7$)

They are:

- Mathematically related
- Generated together
- But not reversible without secret information

This here ladies and gentlemen is the core of RSA. Everything we've done here scales directly to real-world RSA. The only difference is key size.

In practice:

- p and q are enormous
- n is thousands of bits long
- Reversing the process becomes computationally infeasible

The *structure* stays exactly the same.

Encrypting and decrypting with our tiny RSA system

At this point we have a working, though completely insecure, RSA key pair:

- *Public key*:
($n = 33$, $e = 3$)
- *Private key*:
($n = 33$, $d = 7$)

Now let's actually *use* it.

Step 6 – Choose a message

In real systems, messages are converted into numbers before encryption. For our example, we'll simply pick a small number:

$m = 4$

This number must be smaller than n (33), which it is.

Why the message must be smaller than n ? In RSA, all operations take place within the numerical space defined by n . The modulus n acts as the upper boundary of that space, you cannot cross that boundary. If a message were equal to or larger than n , the modulo operation would immediately reduce it back into that space, effectively changing the message before encryption even begins. In other words: values outside the range are wrapped around and lose their original meaning, it breaks.

By ensuring that the message is smaller than n , RSA guarantees that the encryption and decryption process operates on a well-defined, reversible value. In real-world systems, this is handled automatically by encoding and padding schemes, but the principle remains the same.

Step 7 – Encrypt using the public key

Encryption in RSA looks like this:

$$\text{ciphertext} = m^e \bmod n$$

Substituting our values:

$$\text{ciphertext} = 4^3 \bmod 33$$

Calculate step by step:

$$4^3 = 64$$

$$64 \bmod 33 = 31$$

So the encrypted message is:

$$c = 31$$

This operation uses only public information:

- The message
- The public exponent
- The public modulus

Anyone can do this, because it's known by everyone, and that's the entire idea behind it.

Step 8 – Decrypt using the private key

Now we reverse the process using the private key:

$$\text{plaintext} = c^d \bmod n$$

Substituting again:

$$\text{plaintext} = 31^7 \bmod 33$$

At first glance, this looks easy right! Open your Windows Calculator (assuming here you're using Windows, but any calculator with the required functions will do), switch to the scientific calculator, and do the math. For small numbers this is perfect to calculate. Now imagine this with large prime numbers.... It's very very compute intense. But modular arithmetic allows a very important simplification, well I needed to read it a couple of times before I got the essence.... But it can be very useful to understand the concept.

A useful trick: simplify early and often

Notice this:

$$31 \equiv -2 \pmod{33}$$

Because:

$$31 = 33 - 2$$

Working with -2 is much easier than working with 31. Why? Because it's smaller, remember that smaller numbers are easier to compute, that's the idea.

So we rewrite:

$$31^7 \bmod 33 = (-2)^7 \bmod 33$$

Now calculate:

$$(-2)^7 = -128$$

Apply the modulus:

$$-128 \bmod 33 = 4$$

And we're back where we started:

$$m = 4$$

The original message is recovered. Cue the dramatic music, we've "*cracked the code*"! Well, not really. We didn't break anything, we just followed the math. But it does feel pretty awesome.

Why this doesn't scale by hand, and that's the point

In real RSA:

- n is 2048 (hopefully) bits or larger
- p and q are humongous prime numbers
- $\phi(n)$ is never revealed
- d cannot be computed without it

You would never compute this manually, and you don't need to. The important takeaway is *not* the arithmetic. *It's the structure*. Once you understand that structure, concepts like:

- Key length
- Factorization attacks
- Public vs private key leakage
- Why RSA becomes slow at scale

all start to make sense naturally.

A subtle but crucial insight

RSA does *not* rely on hiding information in the algorithm.

Everything we did:

- Is publicly known

- Well-documented
- Mathematically sound

The only thing protecting the system is:

- The secrecy of p , q , and d
- The practical impossibility of reversing n into its original primes

This is a recurring theme in cryptography:

Security comes from hard problems, not secret tricks.

Where this leaves us

At this point, we've:

- Built a complete RSA system
- Used it to encrypt and decrypt a message
- Seen exactly where the asymmetry comes from

What remains is to connect this understanding to the real world. In the next section, we'll look at:

- How key length relates to bit size
- Why RSA-1024 is considered broken
- Why modern systems increasingly move away from RSA entirely

Practical example: HTTPS

Let's dig into something we use every day, you even used it when visiting this blog! When you open a website over HTTPS, two types of cryptography are used, each with a very specific role.

The connection starts with asymmetric cryptography. The server presents its public key as part of its TLS certificate. This allows the client to verify the server's identity and securely exchange key material, even though no shared secret exists yet. At this stage, asymmetric cryptography solves two problems at once:

authenticity and secure key exchange over an untrusted network.

Once the handshake is complete, both sides derive a shared session key. From that moment on, the connection switches to symmetric cryptography. Symmetric algorithms are significantly faster and more efficient, making them suitable for encrypting all actual application data flowing through the connection.

Modern protocols such as TLS 1.3 take this one step further by enforcing Perfect Forward Secrecy (PFS). Session keys are generated using ephemeral key exchange mechanisms and are never derived from long-term private keys. Even if a server's private key is compromised later, previously captured traffic cannot be decrypted.

Remember: *Asymmetric cryptography establishes trust and secrecy. Symmetric cryptography carries the conversation.*

Pause here (on purpose)

If this is the first time RSA has actually *felt* understandable, that's normal. Most explanations skip straight from formulas to conclusions. This one intentionally didn't. Take a moment to let this settle, because everything that comes next builds on this foundation.

What “2048-bit” actually means (without switching to math mode)

When people talk about RSA key sizes, they usually say things like:

“2048-bit RSA”

“1024-bit is broken”

“4096-bit is more secure”

At first glance, this sounds like it refers to *precision* or *complexity*. In reality, it's much simpler.

A bit does not describe what a number is. It describes how large it can be.

Bits describe size, not value

A bit is a binary choice: 0 or 1.

With:

- 1 bit → 2 possible values
- 2 bits → 4 possible values
- n bits → 2^n possible values

So when we say:

“RSA-2048”

We mean: “The number n fits into a space of roughly 2^{2048} possible values.” And that's a crazy long value. That is not a number you try to “guess”. It defines an astronomically large range. Don't believe me, open your calculator again and see the results of 2^{2048} . In decimal terms:

2048 bits $\approx$ 600+ decimal digits

That's why key size isn't about *trying all options*. RSA is never attacked by brute force.

The real problem RSA relies on

RSA security does not come from the size of the key space alone. It relies on one specific hard problem:

Given n , can you efficiently find the two prime numbers p and q that were multiplied to create it?

As long as:

- n is large enough
- p and q are randomly chosen
- And no shortcuts exist

This problem is computationally infeasible, yet not impossible, given time, a lot of time it can be done, but not in any lifetime of you and me. Key size determines how hard that problem is, not how many keys exist.

The future of RSA

Once you understand how RSA works, it stops feeling mysterious, but it also starts to feel a bit heavy. RSA was designed to solve a very specific problem, how to exchange secrets and establish trust over an untrusted network. It does that elegantly, but at a cost. RSA operations rely on very large numbers and expensive calculations, which makes them unsuitable for encrypting large amounts of data. That was never a mistake in the design, RSA was meant as a bootstrap mechanism, not a workhorse.

What makes RSA increasingly uncomfortable in practice is its fragility. When things go wrong, they tend to go wrong completely. A leaked private key, weak randomness during key generation, or poorly generated primes don't weaken the system, they break it outright. The algorithm remains mathematically sound, but real-world implementations are often far less forgiving.

Over time, another pressure has emerged: human progress. RSA-1024 didn't become insecure because computers suddenly became fast enough to try every possibility. It became insecure because researchers got better at understanding the structure of the problem itself. Smarter factorization techniques shifted what was once considered infeasible into the realm of the practical, at least for well-funded state attackers. Please don't have the illusion that your home pc can calculate the many possibilities anytime soon.

RSA still works, and it still does exactly what it was designed to do. But increasing key sizes comes with real trade-offs in performance, complexity, and operational risk. Modern systems increasingly prefer cryptographic approaches that achieve the same goals with smaller keys, fewer brittle assumptions, and better efficiency, not because RSA failed, but because we've learned to do better.

“Cryptography rarely breaks because computers get faster, it breaks because humans get smarter.”

What's in store for the near future?

RSA is not the end of the story. Modern systems increasingly rely on elliptic-curve cryptography (ECC), which achieves the same goals as RSA with much smaller keys and better performance, your mobile phone for example loves smaller key sizes, less processing power means longer battery life. At the same time, researchers are actively preparing for a future where quantum computers may change which mathematical problems are considered hard, giving rise to what we now call post-quantum cryptography.

Those topics deserve their own deep dives. For now, the important takeaway is this: cryptography is not static. Algorithms come and go, assumptions evolve, and what is considered secure today may not be tomorrow.

Understanding RSA isn't about clinging to the past, it's about learning how cryptographic systems are designed, so you can reason about whatever comes next.

Understanding beats memorizing

For a long time, I treated cryptography as something I *used*, but didn't always fully *feel*. I knew the concepts, the terminology, and the standards, yet parts of it still felt abstract, especially when explanations jumped straight to formulas and proofs.

Writing this article forced me to slow down and revisit the basics. Not to simplify them away, but to really understand what is happening under the hood. And interestingly, once I approached cryptography as a system rather than a collection of equations, everything started to make more sense.

Walking through a tiny RSA example, step by step, wasn't about learning RSA. It was about understanding why public and private keys work at all, why key size matters, and why certain cryptographic designs age better than others. Those insights are far more useful than memorizing algorithms.

Cryptography doesn't usually fail because the math is wrong. It fails because assumptions break, keys leak, randomness is weak, or systems are misused. Understanding the structure helps you spot those failure modes long before they turn into incidents.

If this post helped make cryptography feel a little less like “math you’re supposed to trust” and a little more like a system you can reason about, then it served its purpose.

You don’t need to be a math wizard to understand cryptography. You just need a good mental model.

As always, I hope this has given you more insights into the basics of cryptography. It’s just a beginning, but this can help you in understanding what’s going on underneath the hood! If you have any comments, remarks or simple thank you, please let me know.... until next time!